

The WITS Geeks

Design Documentation

Introduction

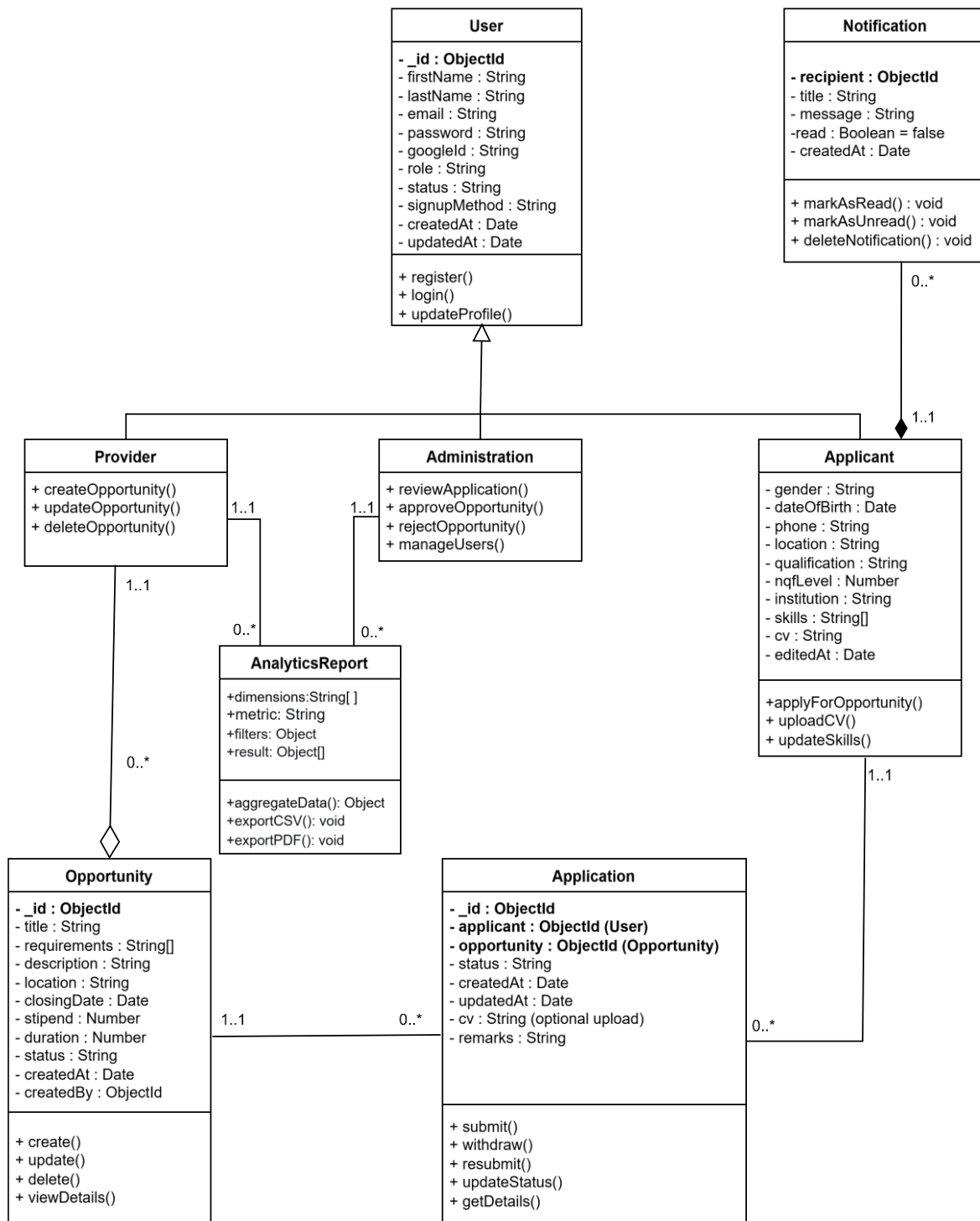
The following section presents the design documents that supported the development of the system. These UML diagrams and design artefacts were used to model the structure, behaviour, and interactions within the system during the design phase. They assisted in improving system understanding, planning implementation, and ensuring that requirements were accurately represented.

The rest of the UMLs are in the Documents in their respective sprints.

Class Diagram

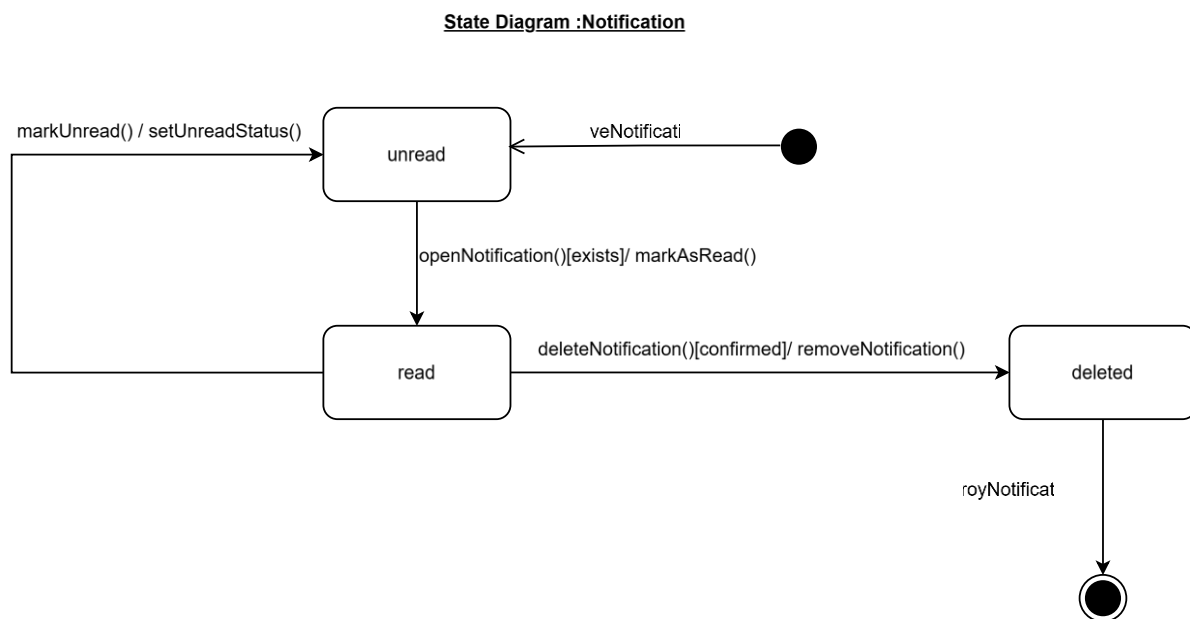
- **User** – stores all account and profile data, with roles (Applicant, Provider, Admin) matching the platform's permissions
- **Opportunity** – represents learnership listings posted by providers
- **Application** – tracks the full lifecycle (Pending → Shortlisted → Rejected) between an applicant and an opportunity
- **Notification** – supports real-time alerts for status changes and new opportunities
- **AnalyticsReport** – new transient class for custom reports; pulls data from User, Application, and Opportunity without cluttering the core models
- Admins and Providers can create many reports, but each report belongs to exactly one user (one-to-many)
- The relationships are straightforward – users interact with opportunities and applications, applications link a user to an opportunity, and the report sits on the side, just reading data when needed
- Keeping the analytics separate keeps the design clean and easy to maintain

Class Diagram: SA Learnerships and Skills Development Portal



Notification state diagram

Notification state diagram – showed the simple lifecycle of a notification: created as Unread, then moving to Read once the user opens it. It kept the notification module tight – no extra states, just what’s needed.



Sequence Diagram

- The sequence diagram laid out the exact flow from the moment an applicant clicks “Apply” to the moment the notification is sent
- It showed the handoff between the frontend, the API, and the database, so we knew exactly where authentication, validation, and data insertion belonged
- We caught the need for duplicate checks and profile-completeness validation before writing any backend code
- It made the order of operations clear: validate first, create the application, then trigger the notification – nothing was left out
- Having this diagram early kept the feature tight and avoided confusion when we later implemented shortlisting and rejection flows

Sequence Diagram Apply for Job Opportunity

